

```
BankDB.sql X + I/O AI Agent 176 ms
CREATE TABLE Account (
  FOREIGN KEY (Customer_ID)
  REFERENCES Customer(Customer_ID)
);

CREATE TABLE Bank_Transaction (
  Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
  Account_No INT,
  Transaction_Type VARCHAR(20),
  Amount DECIMAL(12,2),
  Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (Account_No)
  REFERENCES Account(Account_No)
);

CREATE TABLE Loan (
  Loan_ID INT PRIMARY KEY,
  Customer_ID INT,
  Loan_Type VARCHAR(30),
  Loan_Amount DECIMAL(12,2),
  Interest_Rate DECIMAL(5,2),

  FOREIGN KEY (Customer_ID)
  REFERENCES Customer(Customer_ID)
);

SHOW TABLES;
```

STDIN
Input for the program (Optional)

Output:

Tables_in_sandbox_db
account
bank_transaction
customer
loan

```
BankDB.sql X + I/O AI Agent 238 ms
VALUES
(10001, 101, 'Savings', 50000, 'Hyderabad'),
(10002, 102, 'Savings', 75000, 'Vijayawada'),
(10003, 103, 'Current', 120000, 'Bangalore'),
(10004, 104, 'Savings', 45000, 'Chennai'),
(10005, 105, 'Current', 90000, 'Hyderabad');

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 10000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000);

INSERT INTO Loan
(Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
VALUES
(501, 101, 'Home Loan', 5000000, 7.5),
(502, 102, 'Education Loan', 1000000, 6.5),
(503, 103, 'Car Loan', 800000, 8.2),
(504, 104, 'Personal Loan', 500000, 10.5);

SELECT * FROM Customer;
SELECT * FROM Account;
SELECT * FROM Bank_Transaction;
SELECT * FROM Loan;
```

STDIN
Input for the program (Optional)

Output:

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad

```
BankDB.sql X + I/O AI Agent 238 ms
VALUES
(10001, 101, 'Savings', 50000, 'Hyderabad'),
(10002, 102, 'Savings', 75000, 'Vijayawada'),
(10003, 103, 'Current', 120000, 'Bangalore'),
(10004, 104, 'Savings', 45000, 'Chennai'),
(10005, 105, 'Current', 90000, 'Hyderabad');

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 10000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000);

INSERT INTO Loan
(Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
VALUES
(501, 101, 'Home Loan', 5000000, 7.5),
(502, 102, 'Education Loan', 1000000, 6.5),
(503, 103, 'Car Loan', 800000, 8.2),
(504, 104, 'Personal Loan', 500000, 10.5);

SELECT * FROM Customer;
SELECT * FROM Account;
SELECT * FROM Bank_Transaction;
SELECT * FROM Loan;
```

STDIN
Input for the program (Optional)

Output:

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-13 09:04:
2	10002	DEPOSIT	15000.00	2026-08-13 09:04:
3	10003	WITHDRAW	20000.00	2026-08-13 09:04:
4	10004	DEPOSIT	5000.00	2026-08-13 09:04:
5	10005	WITHDRAW	10000.00	2026-08-13 09:04:

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
501	101	Home Loan	5000000.00	7.50
502	102	Education Loan	1000000.00	6.50
503	103	Car Loan	800000.00	8.20
504	104	Personal Loan	500000.00	10.50

BankDB.sql

1

2

3

4

5

6

7

8

9

10

11

DELIMITER //

CREATE PROCEDURE GetAllCustomers()

BEGIN

SELECT * FROM Customer;

END //

DELIMITER ;

CALL GetAllCustomers();

I/O

AI Agent

156 ms

STDIN

Input for the program (Optional)

Output:

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

Dark Wiki MySQL

BankDB.sql

1

2

3

4

5

6

7

8

9

10

11

12

13

14

DELIMITER //

CREATE PROCEDURE GetAccountDetails(
IN p_Account_No INT
)

BEGIN

SELECT *
FROM Account
WHERE Account_No = p_Account_No;

END //

DELIMITER ;

CALL GetAccountDetails(10001);

I/O

AI Agent

160 ms

STDIN

Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad

Dark Wiki MySQL

BankDB.sql

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

DELIMITER //

CREATE PROCEDURE GetCustomerAccounts(
IN p_Customer_ID INT
)

BEGIN

SELECT
C.Customer_ID,
C.Customer_Name,
A.Account_No,
A.Account_Type,
A.Balance,
A.Branch
FROM Customer C
JOIN Account A
ON C.Customer_ID = A.Customer_ID
WHERE C.Customer_ID = p_Customer_ID;

END //

DELIMITER ;

CALL GetCustomerAccounts(101);

I/O

AI Agent

156 ms

STDIN

Input for the program (Optional)

Output:

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	10001	Savings	50000.00	Hyderabad

Dark Wiki MySQL

BankDB.sql

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

DELIMITER //

CREATE PROCEDURE DepositMoney(

IN p_Account_No INT,

IN p_Amount DECIMAL(12,2)

)

BEGIN

UPDATE Account

SET Balance = Balance + p_Amount

WHERE Account_No = p_Account_No;

END //

DELIMITER ;

CALL DepositMoney(10001, 5000);

SELECT *

FROM Account

WHERE Account_No = 10001;

I/O

AI Agent

225 ms

STDIN

Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	55000.00	Hyderabad

BankDB.sql

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

DELIMITER //

CREATE PROCEDURE WithdrawMoney(

IN p_Account_No INT,

IN p_Amount DECIMAL(12,2)

)

BEGIN

UPDATE Account

SET Balance = Balance - p_Amount

WHERE Account_No = p_Account_No;

END //

DELIMITER ;

CALL WithdrawMoney(10001, 3000);

SELECT *

FROM Account

WHERE Account_No = 10001;

I/O

AI Agent

149 ms

STDIN

Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	47000.00	Hyderabad

BankDB.sql

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

DELIMITER //

CREATE TRIGGER CheckBalance

BEFORE UPDATE ON Account

FOR EACH ROW

BEGIN

IF NEW.Balance < 0 THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT =

'Transaction failed: Insufficient balance';

END IF;

END //

DELIMITER ;

UPDATE Account

SET Balance = Balance - 60000

WHERE Account_No = 10001;

I/O

AI Agent

Error

STDIN

Input for the program (Optional)

Output:

ERROR 1644 (45000) at line 92: Transaction failed: Insufficient balance

```
BankDB.sql x + I/O AI Agent Error
1
2
3 DELIMITER //
4
5 CREATE TRIGGER CheckTransactionAmount
6 BEFORE INSERT ON Bank_Transaction
7 FOR EACH ROW
8 BEGIN
9     IF NEW.Amount <= 0 THEN
10         SIGNAL SQLSTATE '45000'
11         SET MESSAGE_TEXT =
12             'Transaction amount must be greater than zero';
13     END IF;
14 END //
15
16 DELIMITER ;
17
18 INSERT INTO Bank_Transaction
19 (Account_No, Transaction_Type, Amount)
20 VALUES
21 (10001, 'DEPOSIT', -5000);
```

STDIN

Input for the program (Optional)

Output:

ERROR 1644 (45000) at line 92: Transaction amount must be greater than zero

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 209 ms
15 CREATE TRIGGER TransactionAudit
16 AFTER INSERT ON Bank_Transaction
17 FOR EACH ROW
18 BEGIN
19     INSERT INTO Transaction_Audit
20     (
21         Transaction_ID,
22         Account_No,
23         Transaction_Type,
24         Amount
25     )
26     VALUES
27     (
28         NEW.Transaction_ID,
29         NEW.Account_No,
30         NEW.Transaction_Type,
31         NEW.Amount
32     );
33 END //
34
35 DELIMITER ;
36
37 INSERT INTO Bank_Transaction
38 (Account_No, Transaction_Type, Amount)
39 VALUES
40 (10001, 'DEPOSIT', 2500);
41
42 SELECT * FROM Transaction_Audit;
```

STDIN

Input for the program (Optional)

Output:

Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
1	6	10001	DEPOSIT	2500.00	2026-08-

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 292 ms
5 AFTER INSERT ON Bank_Transaction
6 FOR EACH ROW
7 BEGIN
8     IF NEW.Transaction_Type = 'DEPOSIT' THEN
9
10         UPDATE Account
11         SET Balance = Balance + NEW.Amount
12         WHERE Account_No = NEW.Account_No;
13
14     ELSEIF NEW.Transaction_Type = 'WITHDRAWAL' THEN
15
16         UPDATE Account
17         SET Balance = Balance - NEW.Amount
18         WHERE Account_No = NEW.Account_No;
19
20     END IF;
21 END //
22
23 DELIMITER ;
24
25 INSERT INTO Bank_Transaction
26 (Account_No, Transaction_Type, Amount)
27 VALUES
28 (10001, 'DEPOSIT', 5000);
29
30 SELECT *
31 FROM Account
32 WHERE Account_No = 10001;
```

STDIN

Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	55000.00	Hyderabad

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent Error
4
5 CREATE TRIGGER PreventInsufficientWithdrawal
6 BEFORE INSERT ON Bank_Transaction
7 FOR EACH ROW
8 BEGIN
9     DECLARE CurrentBalance DECIMAL(12,2);
10
11     SELECT Balance
12     INTO CurrentBalance
13     FROM Account
14     WHERE Account_No = NEW.Account_No;
15
16     IF NEW.Transaction_Type = 'WITHDRAW'
17     AND NEW.Amount > CurrentBalance THEN
18
19         SIGNAL SQLSTATE '45000'
20         SET MESSAGE TEXT =
21         'Withdrawal failed: Insufficient balance';
22
23     END IF;
24 END //
25
26 DELIMITER ;
27
28 INSERT INTO Bank_Transaction
29 (Account_No, Transaction_Type, Amount)
30 VALUES
31 (10001, 'WITHDRAW', 1000000);
```

STDIN
Input for the program (Optional)

Output:
ERROR 1644 (45000) at line 169: Withdrawal failed: Insufficient balance

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 141 ms
10 BEGIN
16 WHERE Account_No = SenderAccount;
17
18 IF SenderBalance < TransferAmount THEN
19
20     SIGNAL SQLSTATE '45000'
21     SET MESSAGE TEXT =
22     'Transfer failed: Insufficient balance';
23
24 ELSE
25
26     UPDATE Account
27     SET Balance = Balance - TransferAmount
28     WHERE Account_No = SenderAccount;
29
30     UPDATE Account
31     SET Balance = Balance + TransferAmount
32     WHERE Account_No = ReceiverAccount;
33
34     END IF;
35 END //
36
37 DELIMITER ;
38
39 CALL TransferMoney(10001, 10002, 5000);
40 SELECT *
41 FROM Account
42 WHERE Account_No IN (10001, 10002);
```

STDIN
Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 233 ms
1
2
3 DELIMITER //
4
5 CREATE PROCEDURE GetCustomerLoans(
6     IN p_Customer_ID INT
7 )
8 BEGIN
9     SELECT
10         C.Customer_Name,
11         L.Loan_ID,
12         L.Loan_Type,
13         L.Loan_Amount,
14         L.Interest_Rate
15     FROM Customer C
16     JOIN Loan L
17     ON C.Customer_ID = L.Customer_ID
18     WHERE C.Customer_ID = p_Customer_ID;
19 END //
20
21 DELIMITER ;
22
23 CALL GetCustomerLoans(101);
```

STDIN
Input for the program (Optional)

Output:

Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
Ravi Kumar	501	Home Loan	5000000.00	7.50

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 691 ms ✓
1
2 DELIMITER //
3
4 CREATE PROCEDURE HighBalanceAccounts(
5 | IN MinimumBalance DECIMAL(12,2)
6 | )
7 BEGIN
8 | SELECT *
9 | FROM Account
10 | WHERE Balance >= MinimumBalance
11 | ORDER BY Balance DESC;
12 END //
13
14 DELIMITER ;
15
16 CALL HighBalanceAccounts(50000);
```

STDIN
Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore
10005	105	Current	90000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10001	101	Savings	55000.00	Hyderabad

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 191 ms ✓
20
21 BEGIN
22 | SELECT *
23 | FROM Account
24 | WHERE Balance >= MinimumBalance
25 | ORDER BY Balance DESC;
26 END //
27
28 DELIMITER ;
29
30 DELIMITER //
31
32 CREATE PROCEDURE GetBalance(
33 | IN p_Account_No INT,
34 | OUT p_Balance DECIMAL(12,2)
35 | )
36 BEGIN
37 | SELECT Balance
38 | INTO p_Balance
39 | FROM Account
40 | WHERE Account_No = p_Account_No;
41 END //
42
43 DELIMITER ;
44
45 CALL GetBalance(10001, @CurrentBalance);
46
47 SELECT @CurrentBalance;
```

STDIN
Input for the program (Optional)

Output:

@CurrentBalance
55000.00

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 259 ms ✓
241
242 CREATE PROCEDURE GetBalance(
243 | OUT p_Balance DECIMAL(12,2)
244 | )
245 BEGIN
246 | SELECT Balance
247 | INTO p_Balance
248 | FROM Account
249 | WHERE Account_No = p_Account_No;
250 END //
251
252 DELIMITER ;
253
254 DELIMITER //
255
256 CREATE TRIGGER CheckBalance
257 BEFORE UPDATE ON Account
258 FOR EACH ROW
259 BEGIN
260 | IF NEW.Balance < 0 THEN
261 | | SIGNAL SQLSTATE '45000'
262 | | SET MESSAGE_TEXT =
263 | | 'Transaction failed: Insufficient balance';
264 | END IF;
265 END //
266
267 DELIMITER ;
268
269 SHOW TRIGGERS;
```

STDIN
Input for the program (Optional)

Output:

Trigger	Event	Table	Statement
CheckBalance	UPDATE	account	BEGIN IF NEW.Balance < 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance'; END IF; END PreventInsufficientWithdrawal INSERT bank_transaction BEGIN DECLARE CurrentBalance DECIMAL(12,2); SELECT Balance INTO CurrentBalance FROM Account WHERE Account_No = NEW.Account_No; IF NEW.Transaction_Type = 'WITHDRAWAL'

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 181 ms
35
36 CREATE TRIGGER CheckBalance
37 BEFORE UPDATE ON Account
38 FOR EACH ROW
39 BEGIN
40     IF NEW.Balance < 0 THEN
41         SIGNAL SQLSTATE '45000'
42         SET MESSAGE_TEXT =
43         'Transaction failed: Insufficient balance';
44     END IF;
45 END //
46
47 DELIMITER ;
48
49 DELIMITER //
50
51 CREATE PROCEDURE GetBranchAccounts(
52     IN p_Branch VARCHAR(50)
53 )
54 BEGIN
55     SELECT *
56     FROM Account
57     WHERE Branch = p_Branch;
58 END //
59
60 DELIMITER ;
61
62 CALL GetBranchAccounts('Hyderabad');
```

STDIN

Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	55000.00	Hyderabad
10005	105	Current	90000.00	Hyderabad

Dark Wiki MySQL

```
BankDB.sql x + I/O AI Agent 189 ms
274 BEGIN
275     WHERE Branch = p_Branch;
276 END //
277
278 DELIMITER ;
279
280 DELIMITER //
281
282 CREATE PROCEDURE GetCustomerFullDetails(
283     IN p_Customer_ID INT
284 )
285 BEGIN
286     SELECT
287         C.Customer_Name,
288         C.Phone,
289         C.Email,
290         A.Account_No,
291         A.Account_Type,
292         A.Balance
293     FROM Customer C
294     JOIN Account A
295         ON C.Customer_ID = A.Customer_ID
296     WHERE C.Customer_ID = p_Customer_ID;
297 END //
298
299 DELIMITER ;
300
301 CALL GetCustomerFullDetails(101);
```

STDIN

Input for the program (Optional)

Output:

Customer_Name	Phone	Email	Account_No	Account_Type	Balance
Ravi Kumar	9876543210	ravi@gmail.com	10001	Savings	55000.00

Dark Wiki MySQL

BankDB.sql

287 BEGIN

288 SELECT

294 A.Balance

295 FROM Customer C

296 JOIN Account A

297 ON C.Customer_ID = A.Customer_ID

298 WHERE C.Customer_ID = p_Customer_ID;

299 END //

300

301 DELIMITER ;

302

303 DELIMITER //

304

305 CREATE PROCEDURE GetTotalCustomerBalance(

306 IN p_Customer_ID INT

307)

308 BEGIN

309 SELECT

310 Customer_ID,

311 SUM(Balance) AS Total_Balance

312 FROM Account

313 WHERE Customer_ID = p_Customer_ID

314 GROUP BY Customer_ID;

315 END //

316

317 DELIMITER ;

318

319 CALL GetTotalCustomerBalance(101);

ready

I/O AI Agent 279 ms

STDOUT

Input for the program (Optional)

Output:

Customer_ID	Total_Balance
101	55000.00

Dark Wiki MySQL

BankDB.sql

287 BEGIN

288 SELECT

294 A.Balance

295 FROM Customer C

296 JOIN Account A

297 ON C.Customer_ID = A.Customer_ID

298 WHERE C.Customer_ID = p_Customer_ID;

299 END //

300

301 DELIMITER ;

302

303 DELIMITER //

304

305 CREATE PROCEDURE GetTotalCustomerBalance(

306 IN p_Customer_ID INT

307)

308 BEGIN

309 SELECT

310 Customer_ID,

311 SUM(Balance) AS Total_Balance

312 FROM Account

313 WHERE Customer_ID = p_Customer_ID

314 GROUP BY Customer_ID;

315 END //

316

317 DELIMITER ;

318

319 CALL HighBalanceAccounts(100000);

ready

I/O AI Agent 189 ms

STDOUT

Input for the program (Optional)

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore

Dark Wiki MySQL